

Backlog Ultra Détaillé — Sahar

Pour une personne qui part de zéro. Chaque tâche = quelques heures maximum.

Comment utiliser ce document

Ne saute aucune étape. Fais-les dans l'ordre exact. Chaque tâche a une case à cocher et un "c'est terminé quand..." très précis, pour que tu saches exactement quand passer à la suivante. Ne commence jamais une tâche avant d'avoir fini celle d'avant.

PHASE 0 — Installer tous tes outils (à faire aujourd'hui, avant tout code)

Tâche 0.1 — Installer VS Code

1. Va sur <https://code.visualstudio.com/>
2. Télécharge la version pour ton système (Windows/Mac/Linux) et installe-la (clique "Suivant" partout, les réglages par défaut suffisent).

✅ C'est terminé quand : tu peux ouvrir VS Code et voir son interface.

Tâche 0.2 — Installer Git

1. Va sur <https://git-scm.com/downloads>
2. Installe-le (réglages par défaut).
3. Ouvre un terminal (dans VS Code : menu Terminal → Nouveau terminal) et tape :

```
bash
git --version
```

✅ C'est terminé quand : cette commande affiche un numéro de version (ex : `git version 2.43.0`), pas une erreur.

Tâche 0.3 — Créer ton compte GitHub

1. Va sur <https://github.com> et crée un compte si tu n'en as pas.
2. Configure Git avec ton identité (dans le terminal VS Code) :

```
bash
```

```
git config --global user.name "Sahar"
git config --global user.email "ton-email@example.com"
```

✅ **C'est terminé quand :** tu as un compte GitHub actif.

Tâche 0.4 — Choisir ton environnement : Windows + WSL2 (décision finale, ne pas changer)

⚠️ **Si tu as déjà une VM Ubuntu séparée : ignore-la pour ce projet.** WSL2 est plus léger et s'intègre directement à VS Code Windows (tu gardes toutes tes extensions actuelles). C'est la vraie méthode utilisée en entreprise.

Dans PowerShell en administrateur :

```
powershell
wsl --install
```

Redémarre ton PC. Ouvre "Ubuntu" depuis le menu Démarrer, crée un nom d'utilisateur et un mot de passe.

Dans le terminal Ubuntu qui s'ouvre :

```
bash
sudo apt update && sudo apt upgrade -y
```

Dans VS Code, installe l'extension "**Remote - WSL**" (par Microsoft). Ensuite, (Ctrl+Shift+P) → tape "Remote-WSL: Open Folder in WSL" → choisis ton dossier de projet.

✅ **C'est terminé quand :** VS Code affiche "WSL: Ubuntu" en bas à gauche, et ton terminal intégré ((Ctrl+`)) s'ouvre directement en bash Ubuntu.

Tâche 0.5 — Installer Docker Desktop

1. Va sur <https://www.docker.com/products/docker-desktop/>
2. Télécharge et installe la version pour ton système.
3. Lance Docker Desktop, attends que l'icône indique qu'il tourne.
4. Vérifie dans le terminal :

```
bash
docker --version
docker run hello-world
```

✅ **C'est terminé quand :** la commande (docker run hello-world) affiche un message de bienvenue.

Tâche 0.6 — Installer Python (dans le terminal Ubuntu/WSL2)

```
bash

sudo apt install -y python3 python3-pip python3-venv
python3 --version
```

✅ **C'est terminé quand :** un numéro de version Python s'affiche (ex : `Python 3.11.x`).

Tâche 0.7 — Installer les extensions VS Code

Dans VS Code, clique sur l'icône Extensions (carrés empilés, barre de gauche) et installe :

- **Remote - WSL** (Microsoft) — indispensable, c'est elle qui connecte VS Code à ton Ubuntu WSL2
- **Python** (Microsoft)
- **Docker** (Microsoft)
- **GitLens**
- **Error Lens**
- **YAML**
- **Thunder Client** (pour tester des API sans quitter VS Code)

✅ **C'est terminé quand :** ces extensions apparaissent dans ton onglet "Installées".

Tâche 0.8 — Installer ton assistant IA (Ollama + Cline)

💡 **Simplification importante :** si tu as déjà CodeGeeX, Copilot, Codex, Cline installés, ne les désinstalle pas mais **n'utilise qu'un seul assistant actif à la fois** pour éviter la confusion. Recommandation : Cline (connecté à Ollama en local) comme outil principal.

1. Installe Ollama (dans le terminal Ubuntu/WSL2) :

```
bash

curl -fsSL https://ollama.com/install.sh | sh
ollama pull qwen2.5-coder:7b
```

2. Dans VS Code, installe l'extension **Cline**.
3. Dans les réglages de Cline (icône engrenage), choisis le provider **Ollama**, URL `http://localhost:11434`, modèle `qwen2.5-coder:7b`.

(Détails complets et alternatives dans ton fichier "Setup Installation Complete LLM Local Docker RAG".)

✅ **C'est terminé quand :** tu peux écrire une instruction simple dans Cline ("crée un fichier hello.py qui affiche Bonjour") et voir le modèle local répondre.

Tâche 0.9 — Installer Copilot Free en complément (optionnel mais utile)

1. Dans VS Code, cherche l'extension "GitHub Copilot" et installe-la.
2. Connecte-toi avec ton compte GitHub — le palier gratuit s'active automatiquement.

✅ **C'est terminé quand :** en tapant du code Python, tu vois des suggestions grises apparaître.

PHASE 1 — Comprendre et manipuler Docker (bases absolues)

Tâche 1.1 — Comprendre ce qu'est une image et un conteneur

■ **Image :** un modèle figé, comme une recette de cuisine écrite. ■ **Conteneur :** le plat une fois préparé à partir de la recette — tu peux en refaire autant que tu veux à partir de la même image.

Lance :

```
bash
docker images
docker ps -a
```

✅ **C'est terminé quand :** tu sais expliquer à voix haute la différence entre les deux commandes.

Tâche 1.2 — Créer ton premier dossier de projet

```
bash
mkdir plateforme-iot
cd plateforme-iot
mkdir backend
```

✅ **C'est terminé quand :** le dossier `plateforme-iot/backend` existe.

Tâche 1.3 — Écrire un premier script Python

Crée `backend/app.py` :

```
python
print("Ma plateforme démarre")
```

Teste-le :

```
bash

python backend/app.py
```

✅ **C'est terminé quand :** le message s'affiche dans le terminal.

Tâche 1.4 — Écrire ton premier Dockerfile

Crée `backend/Dockerfile` :

```
dockerfile

FROM python:3.11-slim
WORKDIR /app
COPY app.py .
CMD ["python", "app.py"]
```

✅ **C'est terminé quand :** le fichier existe (pas besoin de le lancer encore).

Tâche 1.5 — Construire et lancer ton image

```
bash

docker build -t mon-backend ./backend
docker run mon-backend
```

✅ **C'est terminé quand :** tu vois "Ma plateforme démarre" s'afficher, mais cette fois exécuté DANS un conteneur.

Tâche 1.6 — Créer ton premier docker-compose.yml

Crée `docker-compose.yml` à la racine du projet :

```
yaml

services:
  backend:
    build: ./backend
```

Lance :

```
bash

docker-compose up
```

✅ **C'est terminé quand :** la commande fonctionne et affiche le même message.

PHASE 2 — FastAPI minimal (le squelette du backend)

Tâche 2.1 — Créer un environnement virtuel Python

```
bash

cd backend
python -m venv venv
```

Active-le :

- Windows : `venv\Scripts\activate`
- Mac/Linux : `source venv/bin/activate`

 **Environnement virtuel** : un espace isolé pour les bibliothèques Python de ce projet, pour ne pas tout mélanger avec d'autres projets.

✓ **C'est terminé quand** : ton terminal affiche `(venv)` au début de la ligne.

Tâche 2.2 — Installer FastAPI

```
bash

pip install fastapi uvicorn
pip freeze > requirements.txt
```

✓ **C'est terminé quand** : le fichier `requirements.txt` contient au moins `fastapi` et `uvicorn`.

Tâche 2.3 — Créer une API qui répond "Hello"

Remplace le contenu de `backend/app.py` :

```
python

from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def racine():
    return {"message": "Hello"}
```

Lance :

```
bash
```

```
uvicorn app:app --reload
```

Ouvre ton navigateur sur `http://localhost:8000` puis sur `http://localhost:8000/docs`.

✅ **C'est terminé quand :** tu vois `{"message": "Hello"}` dans le navigateur, et une interface Swagger sur `/docs`.

Tâche 2.4 — Créer la route POST `/analyze` (vide pour l'instant)

```
python

from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Requete(BaseModel):
    url_github: str

@app.post("/analyze")
def analyser(requete: Requete):
    return {"statut": "Projet reçu", "url": requete.url_github}
```

Teste-la avec l'extension **Thunder Client** installée en Phase 0 : nouvelle requête POST vers `http://localhost:8000/analyze` avec le corps JSON `{"url_github": "https://github.com/test/test"}`.

✅ **C'est terminé quand :** Thunder Client affiche `{"statut": "Projet reçu", "url": "..."}.`

Tâche 2.5 — Mettre à jour le Dockerfile pour FastAPI

```
dockerfile

FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

Mets à jour `docker-compose.yml`:

```
yaml
```

```
services:
  backend:
    build: ./backend
    ports:
      - "8000:8000"
```

bash

```
docker-compose up --build
```

✅ **C'est terminé quand :** `http://localhost:8000/docs` fonctionne, mais cette fois le serveur tourne dans Docker.

PHASE 3 — Agent 1 : détection intelligente (sans LLM d'abord !)

💡 **Point important des retours reçus :** ton Agent 1 n'a pas besoin d'IA pour commencer. Un simple programme qui vérifie l'existence de fichiers suffit dans la majorité des cas. On n'utilisera un LLM que si la détection simple échoue.

Tâche 3.1 — Cloner un dépôt GitHub avec Python

bash

```
pip install gitpython
pip freeze > requirements.txt
```

Crée `backend/agent1.py` :

python

```
from git import Repo
import tempfile

def cloner_depot(url_github: str) -> str:
    dossier_temp = tempfile.mkdtemp()
    Repo.clone_from(url_github, dossier_temp)
    return dossier_temp
```

Teste dans un terminal Python (`python`) puis colle le code, ou crée un `test_agent1.py` avec un vrai dépôt GitHub public simple.

✅ **C'est terminé quand :** le dossier temporaire contient bien les fichiers du dépôt (vérifie avec `ls` ou l'explorateur de fichiers).

Tâche 3.2 — Lister les fichiers d'un dossier

python

```
import os

def lister_fichiers(dossier: str) -> list:
    fichiers = []
    for racine, _, noms_fichiers in os.walk(dossier):
        for nom in noms_fichiers:
            fichiers.append(nom)
    return fichiers
```

✅ **C'est terminé quand :** en testant sur le dossier cloné, tu obtiens une vraie liste de noms de fichiers.

Tâche 3.3 — Détecter Zephyr sans IA (règles simples)

python

```
def detecter_framework(fichiers: list) -> dict:
    if "prj.conf" in fichiers and "CMakeLists.txt" in fichiers:
        return {"framework": "Zephyr RTOS", "confiance": "haute"}
    elif "platformio.ini" in fichiers:
        return {"framework": "Arduino/PlatformIO", "confiance": "haute"}
    else:
        return {"framework": "inconnu", "confiance": "basse"}
```

✅ **C'est terminé quand :** cette fonction retourne `"Zephyr RTOS"` sur un vrai projet Zephyr test (cherche-en un simple sur GitHub, ex. un exemple "blinky" Zephyr forké).

Tâche 3.4 — N'utiliser le LLM que si la détection est incertaine

python

```

import anthropic
import json

client = anthropic.Anthropic(api_key="TA_CLE_API")

def analyser_avec_ia_si_besoin(fichiers: list, resultat_simple: dict) -> dict:
    if resultat_simple["confiance"] == "haute":
        return resultat_simple # pas besoin d'appeler le LLM, on économise !

    prompt = f"""
    Voici la liste des fichiers d'un projet embarqué : {fichiers}
    Réponds UNIQUEMENT en JSON : {"framework": "...", "confiance": "moyenne"}
    """

    response = client.messages.create(
        model="claude-sonnet-4-5", max_tokens=300,
        messages=[{"role": "user", "content": prompt}]
    )
    return json.loads(response.content[0].text)

```

✅ **C'est terminé quand :** tu comprends pourquoi cette approche coûte moins cher et est plus fiable qu'un appel LLM systématique.

Tâche 3.5 — Brancher l'Agent 1 à la route FastAPI

```

python

from fastapi import FastAPI
from pydantic import BaseModel
from agent1 import cloner_depot, lister_fichiers, detecter_framework

app = FastAPI()

class Requete(BaseModel):
    url_github: str

@app.post("/analyze")
def analyser(requete: Requete):
    dossier = cloner_depot(requete.url_github)
    fichiers = lister_fichiers(dossier)
    resultat = detecter_framework(fichiers)
    return resultat

```

✅ **C'est terminé quand :** en testant avec Thunder Client sur un vrai dépôt Zephyr, tu reçois `{"framework": "Zephyr RTOS", "confiance": "haute"}`.

Tâche 3.6 — Publier le contrat d'interface pour Ameni

Crée `contracts/agent1_output.json`:

```
json
{
  "framework": "Zephyr RTOS",
  "fichiers_detectes": ["prj.conf", "CMakeLists.txt"],
  "carte_cible": "esp32",
  "protocoles": ["MQTT", "WiFi"]
}
```

Commit et push ce fichier, préviens Ameni dans votre canal commun.

✅ **C'est terminé quand :** le fichier est visible sur GitHub et Ameni a confirmé l'avoir vu.

PHASE 4 — Étendre l'infrastructure Docker

Tâche 4.1 — Ajouter Mosquitto

```
yaml
services:
  backend:
    build: ./backend
    ports:
      - "8000:8000"

  mosquitto:
    image: eclipse-mosquitto:2
    ports:
      - "1883:1883"
```

```
bash

docker-compose up -d
```

✅ **C'est terminé quand :** `docker ps` montre 2 conteneurs actifs (backend + mosquitto).

Tâche 4.2 — Vérifier que Mosquitto reçoit des messages

Dans un terminal, teste avec un conteneur temporaire :

```
bash

docker run --rm --network plateforme-iot_default eclipse-mosquitto mosquitto_pu
```

✓ **C'est terminé quand :** aucune erreur ne s'affiche (le message a été publié).

Tâche 4.3 — Ajouter InfluxDB

yaml

```
influxdb:
  image: influxdb:2.7
  ports:
    - "8086:8086"
  environment:
    - DOCKER_INFLUXDB_INIT_MODE=setup
    - DOCKER_INFLUXDB_INIT_USERNAME=admin
    - DOCKER_INFLUXDB_INIT_PASSWORD=motdepasse123
    - DOCKER_INFLUXDB_INIT_ORG=iot-org
    - DOCKER_INFLUXDB_INIT_BUCKET=capteurs
```

✓ **C'est terminé quand :** `http://localhost:8086` affiche la page de connexion InfluxDB.

Tâche 4.4 — Ajouter Grafana

yaml

```
grafana:
  image: grafana/grafana:latest
  ports:
    - "3000:3000"
```

✓ **C'est terminé quand :** `http://localhost:3000` affiche la page de connexion Grafana.

Tâche 4.5 — Vérifier l'ensemble

bash

```
docker-compose down
docker-compose up -d
docker ps
```

✓ **C'est terminé quand :** 4 conteneurs tournent en même temps sans erreur (backend, mosquito, influxdb, grafana).

PHASE 5 — Agent 4 : Assistant RAG

Tâche 5.1 — Installer les bibliothèques nécessaires

```
bash

pip install chromadb langchain-text-splitters
pip freeze > requirements.txt
```

✅ **C'est terminé quand :** aucune erreur d'installation.

Tâche 5.2 — Créer ton premier document de test

Crée `docs/erreur_test.md`:

```
markdown

# Erreur : west command not found

Cette erreur signifie que l'outil `west` n'est pas installé ou pas dans le PATH
Solution : lancer `pip install west` puis redémarrer le terminal.
```

✅ **C'est terminé quand :** le fichier existe.

Tâche 5.3 — Découper et indexer ce document

```
python

import chromadb
from langchain_text_splitters import RecursiveCharacterTextSplitter

client_chroma = chromadb.PersistentClient(path="./rag_db")
collection = client_chroma.get_or_create_collection("docs_techniques")
splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)

with open("docs/erreur_test.md") as f:
    texte = f.read()

chunks = splitter.split_text(texte)
collection.add(
    documents=chunks,
    ids=[f"test_{i}" for i in range(len(chunks))]
)
print(f"{len(chunks)} morceaux indexés")
```

✅ **C'est terminé quand :** le message affiche au moins 1 morceau indexé, sans erreur.

Tâche 5.4 — Faire une première recherche

```
python

resultats = collection.query(query_texts=["pourquoi west ne fonctionne pas"], n
print(resultats["documents"])
```

✅ **C'est terminé quand :** le texte de ton fichier `erreur_test.md` apparaît dans les résultats.

Tâche 5.5 — Ajouter la génération de réponse avec le LLM

```
python

def demander_a_assistant(question: str) -> str:
    resultats = collection.query(query_texts=[question], n_results=3)
    chunks_pertinents = "\n---\n".join(resultats["documents"][0])

    prompt = f"""
    Extraits de documentation :
    {chunks_pertinents}

    Question : {question}

    Réponds en te basant uniquement sur ces extraits.
    """
    response = client.messages.create(
        model="claude-sonnet-4-5", max_tokens=500,
        messages=[{"role": "user", "content": prompt}]
    )
    return response.content[0].text

print(demander_a_assistant("J'ai une erreur west command not found, que faire ?"))
```

✅ **C'est terminé quand :** la réponse générée mentionne bien `pip install west`.

Tâche 5.6 — Ajouter 3-5 vrais documents supplémentaires

Répète les tâches 5.2-5.3 avec des vraies erreurs rencontrées par l'équipe (Docker, MQTT, Zephyr).

✅ **C'est terminé quand :** tu as au moins 3 documents indexés et l'assistant répond correctement sur chacun.

PHASE 6 — Finitions

Tâche 6.1 — Créer une route FastAPI pour l'Agent 4

```
python

@app.post("/assistant")
def assistant(requete: Requete):
    return {"reponse": demander_a_assistant(requete.url_github)} # renommer le
```

✅ **C'est terminé quand :** testable via Thunder Client.

Tâche 6.2 — Nettoyer et documenter

- Vérifie que `docker-compose up -d` démarre tout sans erreur.
- Ajoute un README avec les instructions d'installation.

✅ **C'est terminé quand :** une personne qui ne connaît pas le projet pourrait le lancer en suivant ton README.

Tâche 6.3 — Relire la Pull Request CI/CD d'Ameni

Comprends chaque étape de son pipeline, pose des questions si besoin.

✅ **C'est terminé quand :** tu peux expliquer à voix haute ce que fait chaque étape du fichier `.github/workflows/ci.yml`.

Coche chaque tâche au fur et à mesure. Si tu bloques plus de 30 minutes sur une tâche, demande à ton assistant IA de t'expliquer l'erreur exacte avant de continuer.